

ControlPlane AI: Real-Time Observability and Intervention Architecture

Implemented via Model Context Protocol (MCP)

Technical Architecture Blueprint

Contents

1	Introduction and Problem Statement	2
2	Glossary of Terms	2
3	System Architecture: The MCP Router Topology	2
4	Multi-Tier Risk Evaluation Engine: Detection Logic	3
4.1	Tier 0: Deterministic & Heuristic (Latency < 2ms)	3
4.2	Tier 1: Fast-Path Neural (Latency < 25ms)	4
4.3	Tier 2: Async Out-of-Band (Non-Blocking)	4
5	Intervention Engine Workflow	4
6	Implementation Blueprint: FastMCP Python Router	5
7	Deployment and Telemetry Strategy	7

1 Introduction and Problem Statement

Recent advances in agentic AI have shifted the focus from standalone Large Language Models (LLMs) to integrated systems that combine LLMs with tools, memory, and other agents[cite: 1]. A fundamental challenge in deploying these systems is the non-deterministic nature of models, which introduces behavioral uncertainty during execution[cite: 1]. Existing evaluation methods rely too heavily on binary task-completion metrics, failing to capture critical deviations such as skipping diagnostic checks, violating safety policies, or making uninformed decisions[cite: 1].

The ControlPlane Checker is a technology layer designed to transition AI oversight from an after-the-fact discovery to real-time intervention. By acting as a Model Context Protocol (MCP) router, it evaluates every AI action across four foundational pillars: LLMs, Memory, Tools, and Environment[cite: 1].

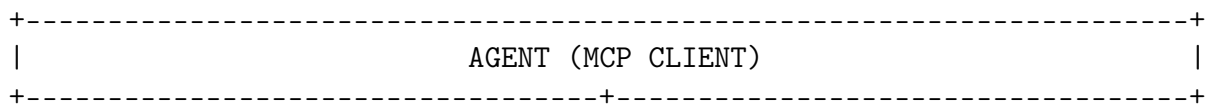
2 Glossary of Terms

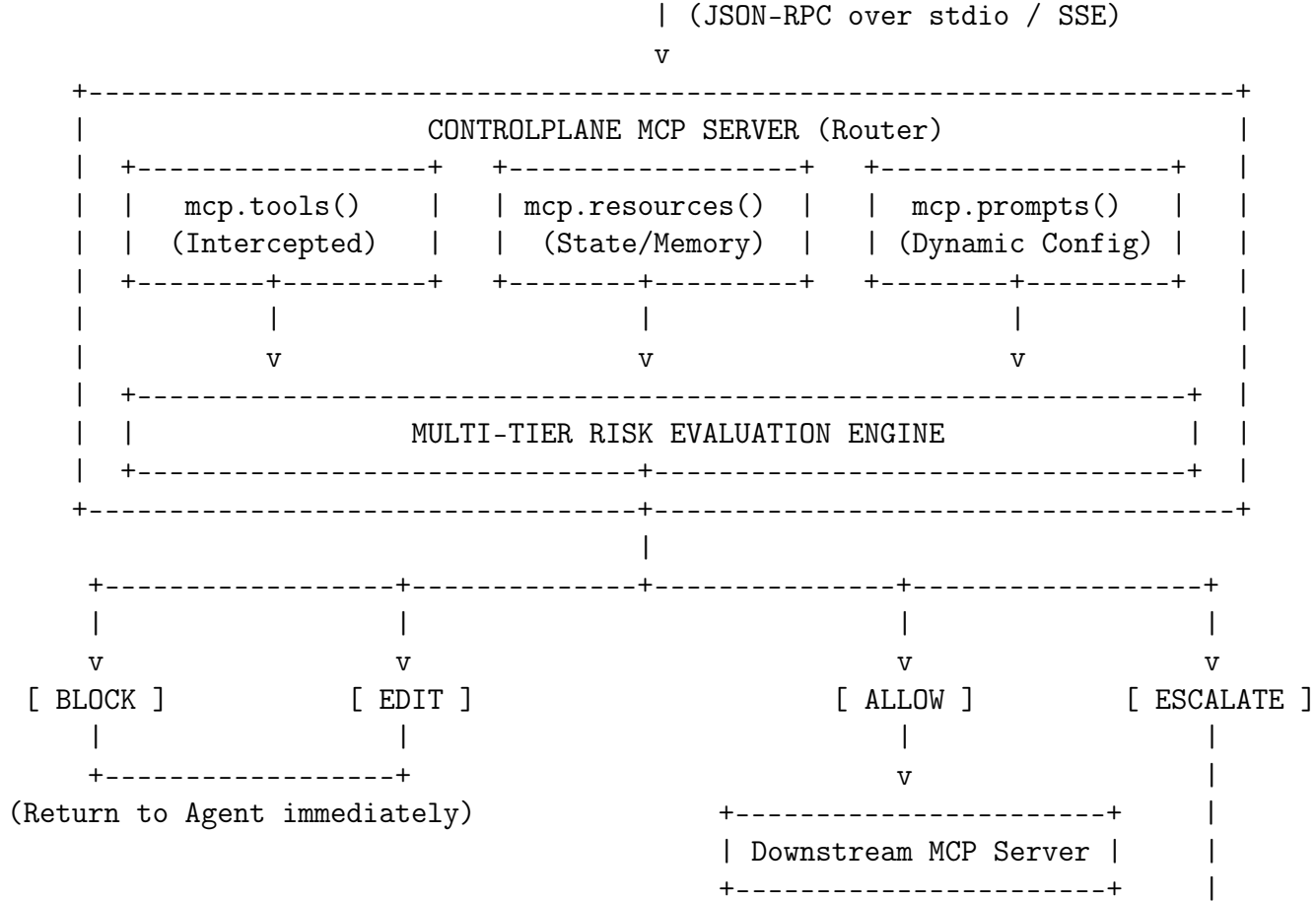
- **Model Context Protocol (MCP):** An emerging open standard that defines how AI agents communicate with external tools and data sources. It standardizes the client-server interaction for tool invocation.
- **ControlPlane:** An observability and intervention layer that continuously observes AI responses in real time across performance, cost, and responsibility dimensions.
- **Agentic AI:** Integrated AI systems equipped with a reasoning component (LLM), memory, and tools for interaction with their environment[cite: 1].
- **SLM (Small Language Model):** Highly optimized, lightweight neural models (e.g., 2-8 billion parameters) used for rapid classification tasks (like toxicity or PII detection) with minimal latency overhead.
- **Guardrails:** Enforced operational boundaries and access controls that define the limits within which an agent operates to prevent harmful outcomes[cite: 1].
- **Diagnostic-Before-Action:** A sequential execution policy where an agent is required to verify system state (e.g., check health) before executing a mutation tool (e.g., delete instance)[cite: 1].

3 System Architecture: The MCP Router Topology

Instead of intercepting raw HTTP traffic using a reverse proxy, the ControlPlane operates natively as a primary MCP Server. The Agent (acting as the MCP Client) connects strictly to this ControlPlane.

When the agent attempts to invoke a tool, the ControlPlane evaluates the request payload, applies deterministic and heuristic guardrails, and subsequently decides whether to block, edit, or forward the request to downstream MCP servers or cloud APIs.





4 Multi-Tier Risk Evaluation Engine: Detection Logic

To evaluate the agent across the Tools, Memory, and Environment pillars without introducing high latency, checks are divided into latency-bound tiers[cite: 1].

4.1 Tier 0: Deterministic & Heuristic (Latency < 2ms)

This tier operates entirely on deterministic algorithms and string manipulation. It acts as the frontline defense to block syntax errors, exact-match policy violations, and looping behaviors before any neural inference occurs.

- **Parameter Mapping Errors (Performance):** Abstract Syntax Tree (AST) parsing and Pydantic schema validation enforce strict type-checking, required fields, and regex bounds on the tool's JSON payload[cite: 1].
- **Workflow Sequencing Skips (Performance):** The engine cross-references the requested tool against a predefined prerequisite map, checking the agent's execution history array to ensure diagnostic tools were invoked first[cite: 1].
- **Infinite Tool Loops (Cost):** The system serializes the tool name and arguments, computes an MD5 hash, and flags a loop if the hash appears ≥ 2 times within a sliding window of the last $N = 3$ execution cycles.
- **PII & Secret Leakage (Responsibility):** An Aho-Corasick automaton and compiled Regular Expressions scan the raw payload for known mathematical patterns

(e.g., the Luhn algorithm for credit cards, standard cloud API credentials).

- **Privilege Escalation (Responsibility):** A deterministic policy matrix matches requested mutation tools and their arguments (e.g., `environment="production"`) against a hardcoded list of protected infrastructure tags[cite: 1].

4.2 Tier 1: Fast-Path Neural (Latency < 25ms)

This tier utilizes Small Language Models (SLMs) and highly optimized tensor operations. It handles semantic and contextual evaluations that cannot be caught by hardcoded rules but require answers fast enough to avoid interrupting the user experience.

- **Confident Hallucinations (Performance):** A quantized cross-encoder computes a semantic entailment score between the retrieved memory context and the agent’s generated action. It flags a hallucination if $P(entrailment) < 0.65$.
- **Toxicity & Prompt Injection (Responsibility):** The input payload is routed through a lightweight sequence classifier (e.g., a quantized DeBERTa-v3 model on ONNX), which outputs categorical probability scores for adversarial or toxic patterns.
- **Memory Entity Inconsistency (Performance):** Token overlap algorithms calculate ROUGE-1 or Token F1-scores between the specific entity IDs in the vector store and the IDs passed to the tool, ensuring the agent is not inventing fictitious resource names[cite: 1].

4.3 Tier 2: Async Out-of-Band (Non-Blocking)

This tier utilizes complex Large Language Models and multi-agent coordination. Because these evaluations take longer (seconds to minutes), they run in the background on message queues (e.g., Kafka) and are used for post-execution auditing, dashboard analytics, and dynamic prompt adjustments rather than real-time blocking.

- **Multi-Hop Memory Gaps (Memory):** An independent LLM-as-a-Judge asynchronously analyzes the agent’s database queries to calculate true retrieval precision and recall, identifying if the agent missed critical context[cite: 1].
- **Sub-optimal Reasoning (Performance):** An auditor Agent-as-a-Judge reviews the complete execution trace, comparing the worker agent’s trajectory against complex, multi-step organizational guidelines to detect logical leaps[cite: 1].
- **Subtle Demographic Bias (Responsibility):** Background workers run counterfactual prompt testing on the agent’s past decisions, altering demographic variables in the logs to statistically measure if the agent alters its tool selection based on protected classes.

5 Intervention Engine Workflow

When the Evaluation Engine detects an anomaly, it returns a descriptive error or takes authoritative action based on severity.

1. **BLOCK (Deterministic Halt):** Applied to irreversible actions violating environment guardrails[cite: 1]. The request is dropped, and a fatal error is returned to the client.
2. **EDIT (In-Flight Correction):** Applied to recoverable errors, such as missing parameters or tool sequencing failures. The system intercepts the failure and injects a descriptive error message guiding the agent on how to correctly proceed.
3. **ESCALATE (Human-in-the-Loop):** Applied to high-impact environment mutations. Execution is paused, state is serialized, and an administrator is notified for manual approval.

6 Implementation Blueprint: FastMCP Python Router

The following is an implementation using the mcp Python SDK demonstrating the ControlPlane logic.

```

1 import hashlib
2 import re
3 import time
4 from typing import Dict, Any, List
5 from mcp.server.fastmcp import FastMCP, Context
6
7 # Initialize the ControlPlane MCP Server
8 mcp = FastMCP("ControlPlane_Guardrail_Router")
9
10 # Global State for Evaluation (Environment & Tools)
11 execution_history: List[str] = []
12 protected_environments = {"prod", "production"}
13 secret_pattern = re.compile(r"(?:AKIA[0-9A-Z]{16}|sk-[a-zA-Z0
14     -9]{32,48})")
15 diagnostic_requirements = {
16     "terminate_instance": ["get_instance_health"]
17 }
18
19 def check_responsibility(params: Dict[str, Any], tool_name: str) -> str
20     | None:
21     """Evaluates safety policies and environment guardrails."""
22     params_str = str(params)
23     if secret_pattern.search(params_str):
24         return "BLOCK: Security Policy Violation - Secret token leakage
25             detected."
26
27     target_env = str(params.get("environment", "")).lower()
28     if target_env in protected_environments and "terminate" in
29         tool_name:
30         return "ESCALATE: Destructive action on production requires
31             manual CAB approval."
32     return None
33
34 def check_performance_and_cost(tool_name: str, params: Dict[str, Any])
35     -> str | None:
36     """Evaluates tool sequencing and loop detection."""
37     serialized = f"{tool_name}:{sorted(params.items())}"
38     current_hash = hashlib.md5(serialized.encode()).hexdigest()

```

```

34     # Loop Detection
35     if execution_history[-3:].count(current_hash) >= 2:
36         return "EDIT: Execution Loop Detected. Analyze prior context
before retrying."
37
38     # Sequence Verification (Diagnostic-before-action)
39     required_diags = diagnostic_requirements.get(tool_name, [])
40     for diag in required_diags:
41         if diag not in [h.split(":")[0] for h in execution_history]:
42             return f"EDIT: Workflow Dependency Broken. Execute '{diag}'
first."
43
44     execution_history.append(f"{tool_name}:{current_hash}")
45     return None
46
47 def execute_downstream_api(tool_name: str, params: Dict[str, Any]) ->
str:
48     """Mocks downstream MCP routing."""
49     return f"Success: {tool_name} executed with {params}"
50
51 @mcp.tool()
52 async def terminate_instance(instance_id: str, environment: str, ctx:
Context) -> str:
53     """Terminates compute instance. Requires environment specification.
"""
54     tool_name = "terminate_instance"
55     params = {"instance_id": instance_id, "environment": environment}
56
57     resp_violation = check_responsibility(params, tool_name)
58     if resp_violation:
59         ctx.info(f"ControlPlane Block: {resp_violation}")
60         return resp_violation
61
62     perf_violation = check_performance_and_cost(tool_name, params)
63     if perf_violation:
64         ctx.info(f"ControlPlane Edit: {perf_violation}")
65         return perf_violation
66
67     return execute_downstream_api(tool_name, params)
68
69 @mcp.tool()
70 async def get_instance_health(instance_id: str, ctx: Context) -> str:
71     """Diagnostic tool to verify instance status."""
72     tool_name = "get_instance_health"
73     params = {"instance_id": instance_id}
74
75     # Register successful diagnostic
76     current_hash = hashlib.md5(f"{tool_name}:{sorted(params.items())}").
encode().hexdigest()
77     execution_history.append(f"{tool_name}:{current_hash}")
78
79     return f"Instance {instance_id} is running normally."
80
81 if __name__ == "__main__":
82     mcp.run()

```

7 Deployment and Telemetry Strategy

To assure observability in production, the architecture integrates OpenTelemetry (OTel). Using the **Context** object provided by the MCP SDK, distributed trace spans are emitted to backends like Grafana or ClickHouse. This fulfills the necessity of tracking an agent's internal states, execution choices, and error interpretations across the system lifecycle. For lower-latency workflows, the MCP Server is run over Standard I/O (stdio), whereas distributed, decoupled setups utilize Server-Sent Events (SSE).